

CPU Scheduling Simulator

APP Project | Project Overview and Technical Summary

This project is a browser-based CPU scheduling simulator designed to help users understand how operating systems schedule processes, compare different scheduling policies, and evaluate performance metrics through interactive visualizations.

1. Project Overview

The application provides a learning-oriented environment where users can create workloads, simulate process execution, and analyze scheduling decisions made by classic and modern algorithms. It combines a graphical user interface with a lightweight backend to support workload generation and workspace/profile management.

2. Objectives

- Visualize CPU scheduling behavior for different process workloads.
- Compare algorithm outcomes such as waiting time, turnaround time, and completion order.
- Teach operating system concepts using a practical simulation experience.
- Support process queue creation, job admission, and performance analytics.
- Provide a smooth front-end experience with accessibility and theme options.

3. Main Features

- Interactive process queue and job queue management.
- Support for algorithms including FCFS, SJF, SRTF, Round Robin, Priority Scheduling, EDF, MLFQ, and CFS.
- Gantt-chart style execution visualization and process lifecycle tracking.
- Workload generation with uniform, Poisson, and exponential distributions.
- Metrics for completion time, waiting time, turnaround time, and scheduling efficiency.
- Ability to import/export workloads and simulation results in JSON/CSV formats.
- Theme switching, accessibility support, and user profile/workspace storage.

4. System Architecture

The project is structured into two main parts: the frontend and the backend. The frontend is built using HTML, CSS, and JavaScript and presents the simulator, job queue, metrics panels, and navigation. The backend is implemented in Python and includes a lightweight HTTP server, API endpoints, workload-generation logic, and optional FastAPI-based services. SQLite storage is used for profiles, workspaces, logs, and simulation-related data.

5. Scheduling Algorithms Included

FCFS	First Come, First Served: processes run in arrival order.
SJF	Shortest Job First: selects the shortest burst next.

SRTF	Shortest Remaining Time First: preemptive version of SJF.
Round Robin	Uses a quantum and cycles through the ready queue.
Priority	Runs the highest-priority process based on configured ordering.
MLFQ	Uses multiple feedback queues to reduce starvation and adapt to behavior.
EDF	Chooses the process with the earliest deadline.
CFS	Fair scheduling based on virtual runtime distribution.

6. Run Instructions

From the project root, start the server with the following command:

```
python Backend/server.py
```

Then open <http://127.0.0.1:8000/> in a browser to use the simulator. The backend also supports FastAPI mode and exposes workload and profile APIs for extended functionality.

7. Team Information

Aditya Kumar Singh	RA2511028020094
Suryanshu Roy	RA2511028020093
Musani Likith Reddy	RA2511028020082

Summary: This project turns theoretical CPU scheduling concepts into an interactive, visual learning tool that makes process behavior and performance trade-offs easy to understand and compare.